

CSCI 236: Data Structures and Algorithms (Fall 2025)

Class Activity

1. HASHING

(A) Imagine you are given the following set of values to insert into a hash table. Assume this hash table:

- Hashes values using the function $h(x) = x$
- Stores each value at $index = h(x) \% table_size$
- Uses **separate chaining** to resolve collisions, adding new values to the **back** of the list - Has an initial internal capacity of 5
- **Triggers a resizing** of the internal array for the next value **after** reaching $\lambda = 1$

Draw the internal state of the hash table as you add the values in the given order. One of these values will trigger a resizing of the array, at this point be sure to transfer the values over so the larger array will represent the final state after adding all values.

values to add: 5, 7, 8, 4, 2, 10, 12, 25, 18

Figure 1 – Internal array of hash table before resizing

0	1	2	3	4

Figure 2 – Internal array of hash table after resizing

0	1	2	3	4	5	6	7	8	9

(B) Imagine a hash table has an initial capacity of 10. During each resizing, the capacity, or *table_size*, of the internal array is doubled. Integer values are inserted into the table using the hash function $h(x) = 5x$. Then each value will be stored at $index = h(x) \% table_size$

- (i) Why is this design suboptimal? Explain briefly.
- (ii) Describe a change that could improve this design and briefly explain how it would impact the runtime.

2. TREES

- (A) Insert the following sequence of values into an empty AVL tree in the given order. 4, 20, 32, 17, 101, 74, 1, 12, 7
- (B) Based on your tree, delete the node, 74. Circle all the nodes in the tree that could take its place without breaking the AVL invariants or moving more than that single node.
- (C) Insert the following sequence of values in the given order, one at a time, into an empty min heap 13, 42, 11, 35, 3, 22, 8, 9